

1. Объяснение базовых понятий: что такое тестирование и почему оно важно

Какие виды тестирования вам знакомы?

- **unit testing (модульное)** – проверка отдельных компонентов (функций, классов) в изоляции.

Примеры: Функции нормализации, препроцессинга данных

- **integration testing (интеграционное)** – тестирование взаимодействия между модулями или системами.

Примеры: пайплайн preprocess → train → predict

- **functional testing (функциональное)** – проверка, работает ли система так, как задумано.

Примеры: Запускаем и получаем какой-то результат. Это ожидаемый результат? При классификации кошек и собак не появилось ли бобр-курв/жирфов

- **end-to-end testing (сквозное)** – тестирование полного workflow приложения, имитирующего действия пользователя.

Примеры: Пользователь заходит на сайт, заходит на страницу личного кабинета, видит уведомления, нажимает по ним, появляется окно, взаимодействует и т.д.

- **system testing (системное)** – проверка всей системы в целом на соответствие требованиям, которые ставились заказчиком.
- **regression testing (регрессионное)** – тестирование после изменений, чтобы убедиться, что новые правки не сломали старый функционал.
- **load testing (нагрузочное)** – оценка производительности при высокой нагрузке.
- **usability testing (юзабилити-тестирование)** – проверка удобства интерфейса для пользователей.
- **security testing/ penetration testing (тестирование безопасности)** – поиск уязвимостей и проверка защиты данных/

Почему тестирование важно?

- **Снижает количество багов** – чем раньше найдена ошибка, тем дешевле её исправить.
- **Повышает надежность** – пользователи получают стабильный продукт.
- **Экономит деньги** – исправление ошибок после релиза обходится дороже, чем на этапе разработки.
- **Защищает репутацию** – плохо протестированное ПО может привести к потере клиентов.

**Уровни тестирования

- **White-Box** – для глубокой проверки кода (разработчики, QA-инженеры).
- **Black-Box** – для проверки функциональности без знания кода (тестировщики, аналитики).

- **Grey-Box** – баланс между ними, полезен при интеграции компонентов.

2. UNIT тестирование

Используется чаще всего библиотека - `pytest` - внешний модуль, или встроенный в питон `unittest`.
Правила:

1. Класс/функции начинаются или заканчиваются словом `test`
2. Для валидации и проверки результатов используется `assert`
3. Один тест - одна функциональность/логика (в идеале)

Детали и фишки которые стоит знать:

1. Моки данных
2. Фикстуры
3. Параметризация
4. Патч любых методов/классов

Примеры unit-тестирования:

1. Простое тестирование

1.1 Тестирование функций

```
import pytest

def test_addition():
    assert 2 + 2 == 4

def test_subtraction():
    assert 5 - 3 == 2
```

1.2 Тестирование с фикстурами

```
import pytest

@pytest.fixture
def my_data():
    return [1, 2, 3]

def test_sum_using_fixture(my_data):
    assert sum(my_data) == 6
```

1.3 Параметризованный тест

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (1, 2, 3),
    (5, 3, 8),
    (0, 0, 0)
])

def test_addition_parametrized(a, b, expected):
    assert a + b == expected
```

1.4 Мокированный тест

Для тестирования функции, которая использует внешний API.

```
from unittest.mock import Mock

def get_weather(api_client, city):
    return api_client.fetch_weather(city)

def test_get_weather():
    mock_api = Mock()
    mock_api.fetch_weather.return_value = "Солнечно"

    result = get_weather(mock_api, "Владивосток")
    assert result == "Солнечно"
    mock_api.fetch_weather.assert_called_with("Владивосток")
```

3. Особенности тестирования в ML

1. Тестирование данных

1.1 Проверка распределений (data drift)

```
import pytest
import numpy as np

def test_data_distribution():
    train_data = np.random.normal(0, 1, 1000)
    test_data = np.random.normal(0, 1, 1000)

    from scipy.stats import ks_2samp
    _, p_value = ks_2samp(train_data, test_data)
    assert p_value > 0.05, "Warning!"
```

1.2 Проверка корректности разметки

```
def test_labels():
    y_train = [0, 1, 0, 1, 2]
    unique_classes = set(y_train)

    assert len(unique_classes) == 3, ""
    assert max(unique_classes) == 2, "Exist label > 2"
```

1.3 Проверка наличия колонок

```
def test_columns_exist(df):
    required_columns = ["age", "income", "target"]
    for column in required_columns:
        assert column in df.columns, f"Column {column} is missing"
```

1.4 Проверка типов данных

```
def test_column_types(df):
    assert df["age"].dtype == "int64", "Column 'age' must be integer"
    assert df["income"].dtype == "float64", "Column 'income' must be float"
```

1.5 Проверка на пропуски (NaN)

```
def test_no_nans_in_critical_columns(df):
    critical_columns = ["age", "income"]
    for column in critical_columns:
        assert not df[column].isnull().any(), f"Column {column} has NaNs"
```

1.6 Проверка среднего и стандартного отклонения

```
def test_mean_in_range(df):
    mean_age = df["age"].mean()
    assert 25 <= mean_age <= 40, f"Mean age {mean_age} is outside expected range"
```

1.7 Проверка на выбросы (IQR метод)

```
def test_no_outliers_iqr(df):
    Q1 = df["income"].quantile(0.25)
    Q3 = df["income"].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df["income"] < lower_bound) | (df["income"] > upper_bound)]
    assert len(outliers) == 0, f"Found {len(outliers)} outliers in 'income'"
```

1.8 Проверка уникальных значений (категориальные данные)

```
def test_valid_categories(df):
    valid_categories = ["A", "B", "C"]
    invalid_categories = set(df["category"].unique()) - set(valid_categories)
    assert not invalid_categories, f"Invalid categories: {invalid_categories}"
```

1.9 Проверка дубликатов

```
def test_no_duplicates(df):
    assert not df.duplicated().any(), "Data contains duplicates"
```

1.10 Проверка логических условий

```
def test_age_positive(df):
    assert (df["age"] > 0).all(), "Age must be positive"
```

1.11 Проверка взаимосвязей колонок

```
def test_income_vs_expenses(df):
    assert (df["income"] >= df["expenses"]).all(), "Income cannot be less than expenses"
```

2. Тестирование пайплайнов

2.1 Тест на отсутствие NaN после обработки

```
def test_preprocessing_pipeline():
    from sklearn.pipeline import Pipeline
    from sklearn.impute import SimpleImputer

    pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='mean'))
    ])

    X = [[1, 2], [np.nan, 3]]
    X_transformed = pipeline.fit_transform(X)

    assert not np.isnan(X_transformed).any(), "NaN не были обработаны!"
```

3. Тестирование моделей

3.1 Тест на минимальную точность

```
def test_model_accuracy():
    from sklearn.metrics import accuracy_score

    y_true = [0, 1, 0, 1]
    y_pred = [0, 1, 1, 1] # Accuracy = 0.75

    assert accuracy_score(y_true, y_pred) > 0.7, "Точность ниже порога"
```

3.2 Тест на входные данные и корректную обработку/ожидаемые ошибки

```
import numpy as np
import pytest
from sklearn.linear_model import LogisticRegression

def test_feature_dimension():
    model = LogisticRegression()
    X_train = np.random.rand(100, 5)
    y_train = np.random.randint(0, 2, 100)

    model.fit(X_train, y_train)

    with pytest.raises(ValueError):
        X_test_wrong_dim = np.random.rand(10, 3)
        model.predict(X_test_wrong_dim)
```

4. Домашнее задание

Написать тесты на этот код:

```
import requests
from collections import Counter

class APIError(Exception):
    pass

class CatFactProcessor:
    def __init__(self):
        self.last_fact = ""

    def get_fact(self):
        try:
            response = requests.get("https://catfact.ninja/fact")
            response.raise_for_status()
            data = response.json()
            self.last_fact = data["fact"]
            return self.last_fact
        except requests.exceptions.RequestException as e:
```

```
        raise APIError(f"Ошибка при запросе к API: {e}") from e

    def get_fact_analysis(self):
        if not self.last_fact:
            return {"length": 0, "letter_frequencies": {}}

        fact_length = len(self.last_fact)
        letter_frequencies = dict(Counter(self.last_fact.lower()))

        return {
            "length": fact_length,
            "letter_frequencies": letter_frequencies,
        }
```

Ожидаемый результат:

1. Присутствуют положительные и отрицательные тесты
2. Протестированы все функции